
MODULE PYTHON

Python pour l'analyse de données

Des bases du langage à l'analyse d'un vrai jeu de données avec pandas, Matplotlib et Seaborn

À la fin du module, vous saurez



Charger, inspecter et nettoyer un jeu de données réel : manquants, doublons, types



Agréger et croiser les données pour répondre à une question métier



Visualiser avec Matplotlib et Seaborn, avec le bon type de graphique

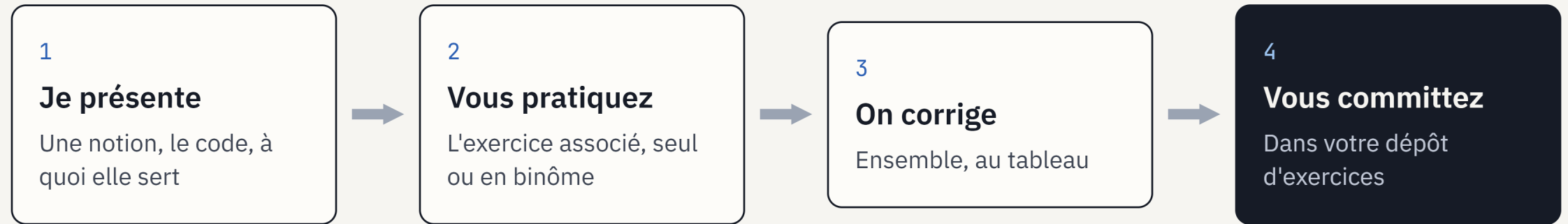


Structurer un travail reproductible, versionné avec Git dès le premier jour



Documenter votre démarche pour qu'un tiers puisse la refaire

Une notion, un exercice



12 exercices, du premier print à la visualisation

Un seul dépôt GitHub, un dossier par exercice

Un mini projet noté sur un jeu de données Kaggle

Installer son poste

- 1 **Python 3.12** depuis python.org. Sous Windows, cocher « Add python.exe to PATH ».
- 2 **VS Code** avec les extensions Python et Jupyter.
- 3 **Git** depuis git-scm.com et un **compte GitHub**.
- 4 **Configurer Git** avec le même email que GitHub.

```
# vérifier dans un terminal
python --version
git --version

# une seule fois
git config --global user.name "Léa Martin"
git config --global user.email "lea@mail.fr"
```

Votre dépôt d'exercices

```
python-exercices/  
├── exercice-01/  
│   └── exercice.py  
├── exercice-02/  
│   └── exercice.py  
├── ...  
├── exercice-06/  
│   └── exercice.ipynb  
├── .gitignore  
├── requirements.txt  
└── README.md
```

requirements.txt : pandas, matplotlib, seaborn, jupyter, une ligne chacun

Sur GitHub : **New repository**, nom python-exercices, public, avec un README et le .gitignore Python.

```
git clone <url-du-depot>  
cd python-exercices  
python -m venv .venv  
# Windows  
.venv\Scripts\activate  
# macOS et Linux  
source .venv/bin/activate  
pip install -r requirements.txt
```

La routine après chaque exercice

```
git status  
git add exercice-03  
git commit -m "Exercice 03 : listes"  
git push
```

Au moins un commit par exercice, plus si vous avancez par étapes

Un message qui dit ce qui a été fait, jamais « update »

Poussez à la fin de chaque séance : votre travail est sauvegardé

L'onglet **Source Control** de VS Code fait la même chose en clics

Votre historique Git montre votre progression. Je le consulte.

Deux façons d'exécuter du Python

Un script .py

Un fichier exécuté en entier, de haut en bas.

`python exercice.py`

Exercices 01 à 05

Un notebook .ipynb

Des cellules exécutées une par une, le résultat s'affiche dessous. Idéal pour explorer des données.

Maj + Entrée

Exercices 06 à 12 et projet

```
print("Bonjour !")  
# une ligne qui commence par # est ignorée
```

print affiche une valeur. C'est votre premier outil pour comprendre ce que fait le code.

Variables et types

```
prenom = "Léa"           # str : du texte
age = 21                  # int : un entier
taille = 1.68             # float : un décimal
inscrite = True           # bool : vrai ou faux

type(age)                # int

int("42")                 # 42
float("3.5")              # 3.5
str(21)                   # "21"
```

Une variable est **un nom posé sur une valeur**.
Le signe = range la valeur de droite sous le nom de gauche.

Le type décide de ce qu'on peut faire : "21" + 1 provoque une erreur

Noms en minuscules avec des _ : **prix_ttc**, pas x ni PrixTTC

Un nom qui dit ce qu'il contient évite un commentaire

Calculer et afficher

Opérateur	Sens	Exemple
+ - * /	Opérations	7 / 2 = 3.5
//	Division entière	7 // 2 = 3
%	Reste	7 % 2 = 1
**	Puissance	2 ** 3 = 8
== !=	Égal, différent	3 == 3
< <= > >=	Comparaisons	age >= 18

Les f-strings

Un f devant les guillemets, les variables entre accolades.

```
prix = 71.6399
print(f"{prenom} a {age} ans")
print(f"Total : {prix:.2f} euros")

# Léa a 21 ans
# Total : 71.64 euros
```

EXERCICE 01

Variables et calculs

exercice-01/exercice.py

Consignes

1. Créer les variables produit = "Clavier", prix_ht = 19.90, quantite = 3 et taux_tva = 0.2
2. Calculer total_ht puis total_ttc
3. Afficher une phrase avec une f-string, le total arrondi à 2 décimales
4. Le prix arrive maintenant sous forme de texte : prix_texte = "19.90". Le convertir et refaire le calcul
5. Commit et push

Résultat attendu

```
3 x Clavier : 71.64 euros TTC
```

Les conditions

```
note = 13.5

if note ≥ 16:
    mention = "Très bien"
elif note ≥ 14:
    mention = "Bien"
elif note ≥ 12:
    mention = "Assez bien"
else:
    mention = "Aucune"

print(mention)  # Assez bien
```

Python teste les conditions **dans l'ordre** et s'arrête à la première vraie.

| **Les deux-points** ouvrent un bloc

| **L'indentation de 4 espaces** délimite le bloc : elle n'est pas décorative

| Combiner avec **and**, **or**, **not** : `age >= 18 and inscrite`

EXERCICE 02

Classer avec des conditions

exercice-02/exercice.py

Consignes

1. À partir d'une variable temperature, afficher « gel » sous 0, « froid » sous 15, « doux » sous 25, sinon « chaud »
2. Tester avec -3, 0, 15 et 31 : vérifier les cas limites
3. Une année est bissextile si elle est divisible par 4 mais pas par 100, ou divisible par 400. Afficher si année l'est
4. Tester avec 2024, 1900 et 2000
5. Commit et push

Résultat attendu

```
-3 °C : gel  
0 °C : froid  
15 °C : doux  
31 °C : chaud  
  
2024 : bissextile  
1900 : non bissextile  
2000 : bissextile
```

Les listes

```
notes = [12, 15.5, 9, 18]

notes[0]          # 12
notes[-1]         # 18
notes[1:3]        # [15.5, 9]
notes.append(14)
len(notes)        # 5
sum(notes) / len(notes)
min(notes), max(notes)
sorted(notes)
```

Une liste range **plusieurs valeurs dans un ordre**, entre crochets.

| **Les positions commencent à 0**, et -1 désigne la dernière

| **[1:3]** prend de la position 1 incluse à 3 exclue

| Une liste se modifie : on ajoute, retire, trie

Les boucles

Parcourir une liste

```
for note in notes:  
    print(note)  
  
for i, note in enumerate(notes):  
    print(f"Note {i + 1} : {note}")
```

Répéter n fois

```
for i in range(3):    # 0, 1, 2  
    print(i)
```

Accumuler un résultat

```
nb_admis = 0  
for note in notes:  
    if note ≥ 10:  
        nb_admis += 1
```

Construire une liste

```
admis = [n for n in notes if n ≥ 10]
```

EXERCICE 03

Une semaine de températures

exercice-03/exercice.py

Consignes

```
temperatures = [12.5, 14, 9.5, 17, 21, 19.5, 11]
```

1. Afficher la moyenne arrondie à 2 décimales, le minimum et le maximum
2. Compter avec une boucle les jours au-dessus de 15 °C
3. Construire la liste en Fahrenheit : $f = c \times 9 / 5 + 32$
4. Afficher chaque jour avec enumerate : « Jour 1 : 12.5 °C »
5. Commit et push

Résultat attendu

```
Moyenne : 14.93
Min : 9.5 / Max : 21
Jours > 15 °C : 3
Fahrenheit : [54.5, 57.2, ...]
Jour 1 : 12.5 °C
Jour 2 : 14 °C
...
```

Les dictionnaires

```
etudiant = {"nom": "Léa", "age": 21}

etudiant["nom"]          # "Léa"
etudiant["ville"] = "Lyon"
"age" in etudiant        # True

for cle, valeur in etudiant.items():
    print(cle, ":", valeur)

promo = [
    {"nom": "Léa", "age": 21},
    {"nom": "Hugo", "age": 23},
]
```

Un dictionnaire associe **une clé à une valeur**, entre accolades.

| On accède par la clé, pas par la position

| Une clé absente provoque une `KeyError`

Une liste de dictionnaires, c'est déjà un tableau de données : une ligne par élément, une colonne par clé.

EXERCICE 04

Le chiffre d'affaires d'un café

exercice-04/exercice.py

Consignes

```
ventes = [  
    {"produit": "café", "prix": 2.5, "quantite": 120},  
    {"produit": "thé", "prix": 2.0, "quantite": 80},  
    {"produit": "jus", "prix": 3.5, "quantite": 45},  
]
```

1. Construire un dictionnaire `ca_par_produit` : produit vers prix × quantité
2. Calculer le chiffre d'affaires total
3. Trouver le produit qui rapporte le plus
4. Commit et push

Résultat attendu

```
{'café': 300.0,  
 'thé': 160.0,  
 'jus': 157.5}  
Total : 617.50 euros  
Meilleur produit : café
```

Les fonctions

```
def moyenne(valeurs):  
    return sum(valeurs) / len(valeurs)  
  
def appliquer_tva(prix_ht, taux=0.2):  
    return prix_ht * (1 + taux)  
  
moyenne([12, 15, 18])    # 15.0  
appliquer_tva(100)       # 120.0  
appliquer_tva(100, 0.055)
```

Une fonction **donne un nom à un traitement** pour le réutiliser.

| **Les paramètres** sont ses entrées, **return** sa sortie

| **Une valeur par défaut** rend un paramètre facultatif

| **Une fonction, une tâche**, un nom qui la décrit par un verbe

| **return n'est pas print** : print affiche, return renvoie une valeur réutilisable

Modules et bibliothèques

Importer

```
import math
math.sqrt(16)          # 4.0
from statistics import median
median([3, 9, 4])      # 4
import pandas as pd
```

Installer une bibliothèque

```
pip install pandas
pip install -r requirements.txt
```

Son propre module

```
# outils.py
def moyenne(valeurs):
    return sum(valeurs) / len(valeurs)

# main.py, dans le même dossier
from outils import moyenne
```

pip s'utilise toujours dans le .venv activé. requirements.txt liste ce dont le projet a besoin.

Lire une erreur

```
Traceback (most recent call last):  
  File "exercice.py", line 3, in <module>  
    total = prix + quantite  
TypeError: unsupported operand type(s) for +: 'float' and 'str'
```

On lit de bas en haut : le type d'erreur, le message, puis la ligne fautive.

Erreur	Cause habituelle
<code>NameError</code>	Variable mal écrite ou pas créée
<code>TypeError</code>	Mélange de types
<code>KeyError</code>	Clé ou colonne absente
<code>IndentationError</code>	Espaces mal alignés

Anticiper une erreur prévisible avec try et except :

```
try:  
    note = float("abc")  
except ValueError:  
    note = None
```

EXERCICE 05

Un module de notes

exercice-05/outils.py + main.py

Consignes

1. Dans outils.py, écrire `convertir_note(texte)` : "12,5" devient 12.5, un texte invalide renvoie None
2. Écrire `moyenne(valeurs)` et `mention(note)`, reprise de l'exercice 02
3. Dans main.py, importer ces fonctions et traiter : `notes_brutes = ["12,5", "15", "abc", "9", "18,25"]`
4. Ignorer les notes invalides, afficher leur nombre, la moyenne et la mention
5. Commit et push

Résultat attendu

```
Notes ignorées : 1
Moyenne : 13.69
Mention : Assez bien
```

Series et DataFrame

```
import pandas as pd

ventes = pd.DataFrame({
    "produit": ["café", "thé", "jus"],
    "prix": [2.5, 2.0, 3.5],
    "quantite": [120, 80, 45],
})

ventes["prix"]          # une Series
ventes["ca"] = ventes["prix"] * ventes["quantite"]
```

	produit	prix	quantite	ca
0	café	2.5	120	300.0
1	thé	2.0	80	160.0
2	jus	3.5	45	157.5

À gauche, **l'index** identifie chaque ligne.

DataFrame : un tableau, des colonnes nommées

Series : une seule colonne, avec son index

Un calcul s'applique à toute la colonne, sans boucle

Charger et typer les données

```
df = pd.read_csv("data/ventes.csv")

# un CSV français : point-virgule, virgule décimale, accents
df = pd.read_csv("data/ventes.csv", sep=";", decimal=",", encoding="latin-1")

# typer dès le chargement
df = pd.read_csv("data/ventes.csv", parse_dates=["date"], dtype={"code_postal": str})

df = pd.read_excel("data/ventes.xlsx")
```

Un code postal lu comme un nombre
perd son zéro : 01000 devient 1000

Des chemins relatifs au projet,
jamais C:\Users\...

Pour les exercices :
`sns.load_dataset("titanic")`

Inspecter avant de toucher

Commande	Ce qu'elle montre
<code>df.shape</code>	Le nombre de lignes et de colonnes
<code>df.head()</code>	Les 5 premières lignes
<code>df.info()</code>	Le type et le nombre de valeurs non nulles par colonne
<code>df.describe()</code>	Moyenne, écart-type, min, quartiles, max des colonnes numériques
<code>df.dtypes</code>	Le type de chaque colonne
<code>df["col"].unique()</code>	Les valeurs distinctes d'une colonne
<code>df["col"].value_counts()</code>	Les effectifs de chaque valeur
<code>df.isna().sum()</code>	Les valeurs manquantes par colonne

Chaque constat d'inspection devient une décision de nettoyage, écrite dans le notebook.

EXERCICE 06

Premier contact avec le Titanic

exercice-06/exercice.ipynb

Consignes

```
import seaborn as sns
titanic = sns.load_dataset("titanic")
```

1. Combien de lignes et de colonnes ?
2. Quel est le type de chaque colonne ? Lesquels vous semblent mal choisis ou redondants ?
3. Combien de valeurs manquantes par colonne ?
4. Quelle est la répartition des passagers par classe et par port ?
5. Écrire 3 constats dans une cellule Markdown, puis commit et push

Pour vérifier

(891, 15)

age	177	manquants
deck	688	manquants
embark_town	2	manquants

Third	491
First	216
Second	184